

Words Before Tokens: Locating a Word’s Information in Byte-Level Indic-Script Language Models, and TokenLift, a Lossless Word-Level Speculative Decoder

Rochit L.

Abstract—Byte-level BPE tokenizers split a Tamil or Hindi word into five to ten tokens, while an English word is usually one. We ask where, inside such a fragmented word, the model’s information about *which* word it is actually lives, and whether that information can be read, moved, or exploited. Measuring per-token surprisal on Qwen3-0.6B and 1.7B over Tamil, Hindi and English Wikipedia, we find that the first token of an Indic word carries almost none of the word’s information—8% for Tamil and 27% for Hindi, against 96% for English—because nearly every word begins with the same shared script bytes; a second model family whose tokenizer fragments Indic far less puts the information back in the first token, showing the effect is tokenizer-driven, not a property of the script. A linear probe on the hidden state *before* the word is written identifies the upcoming word far above dictionary and control baselines, and stays ahead of the written prefix as the word unfolds. Yet this information is not a localized “plan”: activation patching of the pre-word position transfers the donor word in under 8% of matched natural-context pairs, whereas patching the first in-word position dominates it at every matched layer; attention knockout nonetheless shows the pre-word position is a *necessary relay*, costing about two bits and halving correct spelling when blocked. Every finding replicates across two model sizes and a second model family. Guided by this picture, we build a lossless word-level speculative decoder: it drafts whole words and verifies them in one pass, producing token-for-token identical greedy output at up to 2.0 tokens per forward on a laptop. Our strongest drafter, TOKENLIFT, is a word-scoped EAGLE head trained with scheduled-sampling rollouts; it beats the convex drafters at every scale (0.6B, 1.7B, 4B), with a wall-clock payoff that jumps from $\times 1.4$ at 0.6B to $\times 1.7$ – $\times 2.1$ at the larger 1.7B–4B models, and extends to lossless speculative *sampling* that preserves the target’s temperature distribution (2.32 tokens per forward at $\tau=0.7$). TOKENLIFT needs *no* separate draft model: the head is 21 MB (fp16) against a 1.19 GB draft model. A standard same-family draft model (Qwen3-0.6B drafting 4B), where one exists, accepts more per pass (≈ 3.3 vs. 2.2 tokens/forward) but at $56\times$ the auxiliary memory, so we position TOKENLIFT as a *lightweight, single-model accelerator* for when a draft model is unavailable or memory is tight, not as a replacement for draft-model speculative decoding. We release it—head, both decoders, and a headroom diagnostic—as a pip-installable tool, together with all code, data pipeline and results.

Index Terms—large language models, tokenization, interpretability, probing, activation patching, speculative decoding, low-resource NLP, Indic languages.

Independent Researcher, Chennai, India (e-mail: rochitl72@gmail.com; GitHub: github.com/rochitl72/research-paper).

Disclosure of AI use: the author used an AI assistant for code implementation, experiment execution, and drafting. All methods, numbers, and claims were verified by the author, who takes full responsibility for the content.

I. INTRODUCTION

A language model that writes Tamil or Hindi does far more sequential work per word than one writing English. Under the byte-level BPE tokenizer shared by most modern models [1, 2], a Tamil word in Wikipedia text is split by Qwen3’s tokenizer into 9.6 tokens on average and a Hindi word into 4.8, against 1.1 for English. Each of those tokens is one sequential forward pass at generation time, so the same sentence costs several times more latency and money in an Indic script than in English—a disparity documented for tokenization cost and fairness [3, 4].

This paper starts from a simple question about the *internal* side of that disparity. When a model is partway through spelling out a long Indic word, how much of its decision about *which* word this is has already been made, and *where* in the computation does that decision live? A natural hypothesis, which we call *words before tokens*, is that the model commits to a whole word at the position just before it starts writing it, and the remaining tokens are near-deterministic spelling. If true, this would be both scientifically interesting—a word-level plan under a sub-word interface—and practically useful: the spelling tokens could be drafted in parallel and verified in one pass, recovering the latency lost to fragmentation, with no change to the output distribution.

We test this hypothesis directly on Qwen3-0.6B-Base and Qwen3-1.7B-Base [?] over Tamil, Hindi and English Wikipedia, and find that it is half right in an informative way.

- **The first token says almost nothing** (§IV). Because nearly every Tamil word begins with the same token—a space plus the UTF-8 lead bytes shared across the script—only 8% of a Tamil word’s information (in bits) sits in its first token, versus 96% for English. The information is spread across the interior tokens and never falls to zero.
- **The word is readable before it is written** (§V). A linear probe on the pre-word hidden state picks the upcoming word out of 1,000 candidates well above a most-frequent-word baseline and a control-task baseline [5], and is far more accurate on words the model goes on to spell correctly. As the word unfolds, the state stays 10–27 points ahead of what the written prefix alone reveals.
- **But the word is not localized or decided there** (§VI). Activation patching [6, 7] of the pre-word position moves

the next-token decision at most a tenth of the way toward a donor word, while patching the first in-word position reaches 100% at the last layer. Attention knockout shows the other half: the pre-word position is a *necessary* relay—blocking it costs about two bits and halves correct spelling—whose content nonetheless does not carry the choice of word. The word is re-derived from the whole context at each step.

- **A lossless decoder exploits what is there (§VII).** We draft whole words and verify them in one pass, in a loop whose output is provably identical to greedy decoding. Cheap linear “word heads” and Medusa-style token-ahead heads [8, 9] on frozen features already reach 2.0 tokens per forward on Tamil and up to $\times 1.9$ wall-clock on Hindi on a MacBook M1 Pro, and the speedup grows from the 0.6B to the 1.7B model.
- **TOKENLIFT: a word-scoped EAGLE head, released as a tool (§VII-A).** A learned autoregressive draft head [10] scoped to a single word and trained with scheduled-sampling rollouts [11] beats the convex drafters on every language and model size (0.6B, 1.7B, 4B), and its wall-clock payoff jumps from modest at 0.6B ($\times 1.1$ – $\times 1.4$) to large at 1.7B–4B ($\times 1.7$ – $\times 2.1$)—the method matters most at deployable sizes. We extend it to lossless speculative sampling [12, 13]—preserving the target’s temperature- τ distribution, not just its arg-max—where it reaches 2.32 tok/fwd on Tamil at $\tau=0.7$. We release the head, both decoders, and a headroom diagnostic as a `pip`-installable package. To our knowledge this is the first speculative decoder scoped to the word as the unit tokenization fragments, and the first aimed at Indic generation.

Every result is reported for both model sizes; the qualitative picture is identical across scale and the quantitative effects sharpen with it. Our contribution is primarily a measurement and a mechanism—*where* lexical information lives under byte-level tokenization and how it is (and is not) organized—together with a working, laptop-reproducible lossless decoder that puts the mechanism to use. We release the full code, the data pipeline, and the raw results that generate every number and figure here.

II. RELATED WORK

a) Tokenization and language cost.: Byte-level BPE [1, 2] guarantees that any string can be encoded, but allocates its vocabulary budget toward the training mix, so scripts under-represented in that mix fragment into many tokens. Petrov et al. [3] and Ahia et al. [4] quantify the resulting unfairness in sequence length, context usage and API cost across languages. We take this fragmentation as given and ask what it does to the model’s *internal* representation of a word, and whether the extra tokens are cheap to predict.

b) Speculative and parallel decoding.: Speculative decoding [12, 13] accelerates generation by drafting tokens with a cheap proposer and verifying them in one pass of the target model, leaving the output distribution unchanged. Blockwise parallel decoding [14], Medusa [8] and EAGLE [10] replace the separate draft model with extra heads on the target model

itself, and multi-token prediction [9] trains such heads from scratch. Our decoder is in this family, with two differences of emphasis: the draft unit is a whole *word* (a span the tokenizer fragmented), and the heads are fit by convex optimization on frozen features so the whole thing runs on a laptop. Closest in motivation, Yi et al. [15] speed up multilingual inference with language-specific *draft models*; we instead target the within-word fragmentation directly, with per-language word heads on the target model’s own features and no separate draft model. We use these methods as a lens on the representation question rather than to chase state-of-the-art throughput.

c) Probing and causal analysis.: Linear probes [16] measure what is linearly decodable from a hidden state; control tasks [5] guard against probes that succeed by memorization rather than by reading structure, and we adopt their shuffled-label control throughout. To ask whether decodable information is *causal* we use the logit-lens / tuned-lens view of the residual stream [17, 18] and activation patching [6, 7, 19], patching between two natural contexts rather than synthetic edits. The combination of a positive probe, a negative patching result and a positive knockout result is what lets us separate “readable” from “stored here” from “needed here”.

d) Reading and planning future tokens.: Closest to our probe, Future Lens [20] shows that several upcoming tokens can be anticipated from a single hidden state. Wu et al. [21] ask whether this reflects deliberate *pre-caching* of future-token features or mere *breadcrumbs* left by computation useful for the present token, and find natural language modeling is mostly breadcrumbs at small scale. We make both questions concrete for a unit the tokenizer has shattered: whether the identity of a whole multi-token *word* is decodable before it is written in a heavily fragmented script, and—unlike these works—we pair the probe with patching and knockout to test whether that information is a localized stored plan (pre-caching) or re-derived from context at each step (breadcrumbs). For Indic words the answer is the latter, with the pre-word position acting only as a necessary relay. This question is closely related to work on how sub-word models internally reconstruct whole words: ?] show LLMs maintain a word-level “inner lexicon” that detokenizes multi-token words in the middle layers, and ?] find a token-erasure footprint marking where implicit vocabulary items are assembled. Our patching result localizes that assembly causally to the first in-word position rather than the pre-word position, and our decoder turns it to practical use.

III. SETUP AND DEFINITIONS

a) Models.: We study Qwen3-0.6B-Base and Qwen3-1.7B-Base [?]: same tokenizer and depth (28 transformer blocks), hidden size 1024 and 2048 respectively. Both are base (non-instruct) models, run in float32 on a MacBook M1 Pro (16 GB, Metal/MPS). The decoder experiments additionally use Qwen3-4B-Base (hidden size 2560, 36 blocks), run in float16 with 1,800 fit paragraphs per language to fit in memory. All analysis is teacher-forced: we never sample, so every number is deterministic given the text.

Figure B1. One expensive corpus pass feeds every cheap downstream analysis

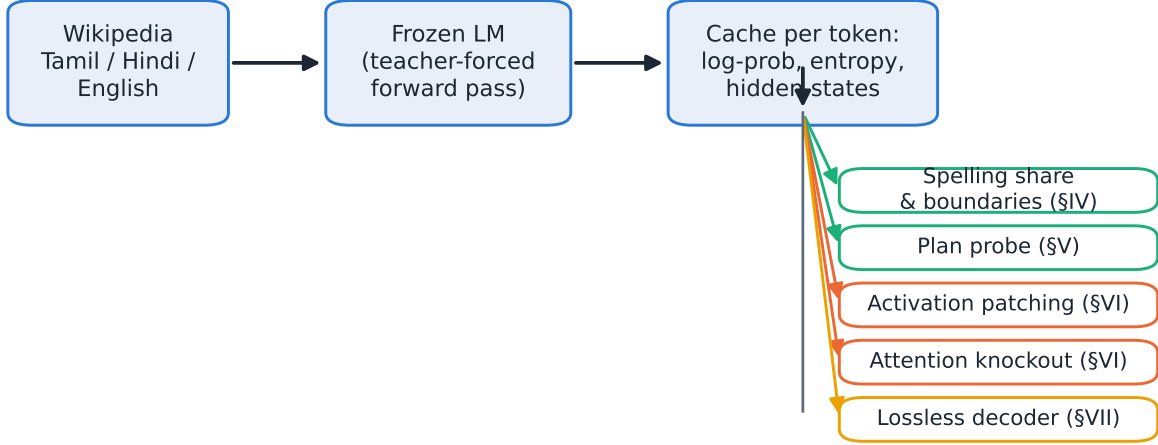


Fig. 1. System overview. A single teacher-forced pass over the corpus caches per-token log-probabilities and hidden states for every multi-token word; all five downstream analyses and the decoder read from that cache, so the expensive work is done once.

TABLE I
CORPUS STATISTICS (IDENTICAL TEXT ACROSS ALL MODELS).

	Tamil	Hindi	English
Fit / eval paragraphs	1500/500	1500/500	1500/500
Words (fit $\cup$ eval)	39,754	74,842	95,672
Tokens per word	9.59	4.82	1.14
Multi-token words	96.4%	98.9%	9.3%

b) *Data.*: For each language we take 6,000 consecutive paragraphs from the `wikimedia/wikipedia 20231101` dumps, filtered to 80–600 characters. Paragraphs 0–3999 form a “fit” pool (for training probes and heads) and 4000–5999 a held-out “eval” pool; experiments use the first 1,500 fit and 500 eval paragraphs, truncated to 256 tokens. Splitting by paragraph range, not at random, keeps whole articles on one side of the split.

c) *Words and boundaries.*: We segment each token sequence into *units* by decoding the bytes of each token and marking a new word unit at a token that begins with a space (or a script word break). A *word* unit with $n > 1$ tokens has one *first token* and $n - 1$ *continuation tokens*. For each continuation token we label the boundary it opens as: **byte**, if it falls inside a single UTF-8 character; **grapheme**, if it falls inside one written syllable (a Unicode grapheme cluster); or **subword**, a split between whole graphemes. Byte- and grapheme-internal splits are artifacts of the encoding; only subword splits are linguistic. Figure 2 illustrates the unit and the three boundary types.

TABLE II
KEY HYPERPARAMETERS. ALL PROBES AND HEADS ARE FIT BY CONVEX OPTIMIZATION ON FROZEN FEATURES.

Component	Setting
Corpus pass	max length 256 tokens; fp32; MPS
Plan probe	1000 classes; AdamW; 40 epochs; 3 seeds
Patching	50–80 natural pairs; min-conf 0.3; layer step 2
Knockout	300 words; 64-token context; 3 seeds
Word head	lexicon 5000; dictionary 30000; ~ 20 epochs
Token-ahead	$K=4$ heads; output vocab 4000
Decoding	12–24 prompts; 32 prompt / 64 new tokens

TABLE III
NOTATION USED THROUGHOUT THE PAPER.

Symbol	Meaning
w ; n	a word; its number of tokens
x_i ; p_i ; s_i	i -th token; $P(x_i x_{<i})$; surprisal $-\log_2 p_i$
$I(w)$	word information, $\sum_i s_i$ (bits)
$\phi(w)$; $\bar{\phi}$	first-token share; corpus aggregate
$SS(\tau)$	spelling share at threshold τ
$h_t^{(\ell)}$	residual-stream state at layer ℓ , position t
W_p	linear probe on the pre-word state
$\Delta_{A \rightarrow B}$	patching effect (normalized logit shift)
t , $d = (d_1 \dots d_k)$	target greedy token; drafted continuation

A. Definitions and notation

Table III collects the symbols used throughout. Let a word w be written as the token sequence $(x_1, \dots, x_n)$, and under teacher forcing let $p_i = P(x_i | x_{<i})$ be the model’s probability of the i -th token and $s_i = -\log_2 p_i$ its surprisal in bits. The

Figure B2. A fragmented Indic word: first token is shared, boundaries are byte / grapheme / subword

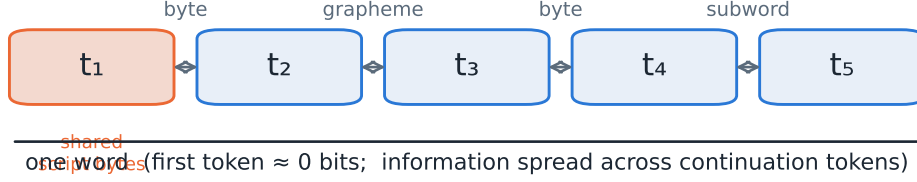


Fig. 2. A fragmented Indic word under byte-level BPE. The first token is a near-deterministic lead shared across the script; the word’s information is spread across continuation tokens, whose boundaries fall inside a UTF-8 character (byte), inside a written syllable (grapheme), or between whole graphemes (subword).

information of the word is

$$I(w) = \sum_{i=1}^n s_i, \quad (1)$$

and its *first-token share* is the fraction of that information carried by the first token,

$$\phi(w) = \frac{s_1}{I(w)}, \quad \bar{\phi} = \frac{\sum_w s_1(w)}{\sum_w I(w)}, \quad (2)$$

reported per language as the corpus aggregate $\bar{\phi}$. A continuation token ($i \geq 2$) is *near-certain* at threshold τ when $p_i \geq \tau$; the *spelling share* is the fraction of all word tokens that are near-certain continuations,

$$SS(\tau) = \frac{\#\{(w, i) : i \geq 2, p_i \geq \tau\}}{\#\{(w, i)\}}, \quad (3)$$

i.e. the share of generation steps a perfectly confident drafter could skip. A low $\bar{\phi}$ means the first token is predictable (shared across the vocabulary) and the word’s identity is resolved by later tokens.

IV. THE FIRST TOKEN OF AN INDIC WORD CARRIES ALMOST NO INFORMATION

Table IV reports the core measurement. Three things stand out, and all three hold for both model sizes (Fig. 3).

Indic words fragment, and the first token is shared. A Tamil word is 9.6 tokens and a Hindi word 4.8, versus 1.1 for English; 96% of Tamil and 99% of Hindi words are multi-token, against 9% of English words. The first-token share is only 8% for Tamil and 25–27% for Hindi, but 95–96% for English. The reason is mechanical: nearly every Tamil word begins with the same token—a leading space followed by the UTF-8 lead bytes common to the whole Tamil Unicode block—so that token is almost free to predict and the word’s identity must be resolved by the tokens that follow. This is the precise internal counterpart of the fragmentation cost of

	Tamil	Hindi	English
Tokens / word	9.59	4.82	1.14
Multi-token words	96.4%	98.9%	9.3%
<i>Spelling share</i> (word tokens near-certain at $p \geq \tau$)			
$\tau = 0.5$	46.4 / 53.5	43.7 / 50.1	7.5 / 8.5
$\tau = 0.9$	27.6 / 35.1	28.5 / 35.9	5.2 / 6.3
$\tau = 0.99$	12.4 / 19.0	14.2 / 21.1	2.6 / 3.7
First-token share	7.7 / 8.2	24.9 / 26.7	94.7 / 95.8

TABLE IV
WHERE A WORD’S INFORMATION LIVES. CELLS WITH TWO NUMBERS ARE QWEN3-0.6B / QWEN3-1.7B. THE FIRST TOKEN HOLDS ALMOST NONE OF A TAMIL OR HINDI WORD’S INFORMATION; CONFIDENCE IN CONTINUATION TOKENS RISES WITH MODEL SIZE.

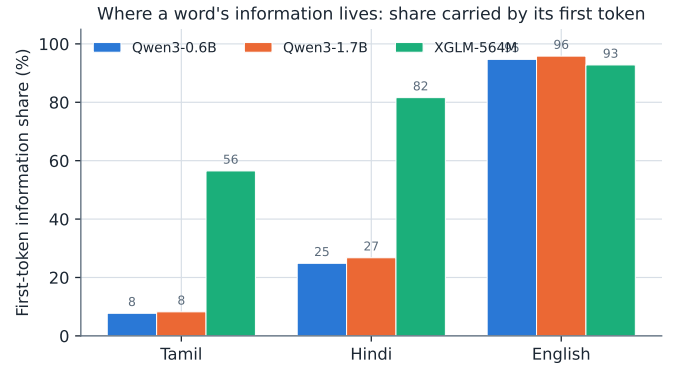


Fig. 3. Share of a word’s information carried by its first token. For Indic scripts under byte-level BPE it is a few percent; for English it is almost all.

Petrov et al. [3]: not only are there more tokens, the *first* one is nearly contentless.

Where the bits are. Figure 4 plots mean surprisal by token position within a word. For Tamil the information is concentrated in the second and third tokens and then settles a little above one bit per token for the rest of the word—it never reaches zero, so most “spelling” tokens still carry a residual

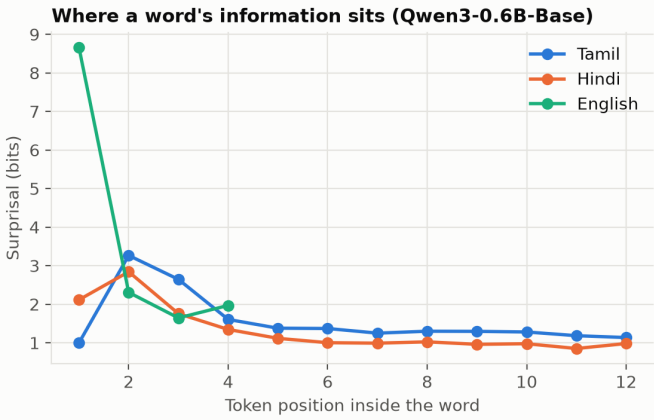


Fig. 4. Mean surprisal (bits) by token position within a word, Qwen3-0.6B. The information of a Tamil/Hindi word is concentrated in its first few interior tokens and then plateaus above zero; an English word is decided at token one.

decision. This already bounds any decoder: the continuation is not free.

Encoding vs. linguistic boundaries. Of Tamil continuation tokens, 36% open a byte-internal boundary and 27% a grapheme-internal one; only 37% are splits between whole graphemes. The encoding-internal splits are the *more* predictable ones (40% near-certain at $\tau = 0.9$ for byte splits vs. 17% for grapheme splits on 0.6B), because completing a half-written character is often forced. A decoder that skips tokens therefore gets its easiest wins on splits that are artifacts of UTF-8, not on linguistic structure.

The spelling share at $\tau = 0.9$ —the fraction of steps a perfectly confident drafter could skip—is 27.6% for Tamil on 0.6B and rises to 35.1% on 1.7B. We set 40% as an a-priori bar for “most of the word is spelling”; neither model clears it, but the gap narrows with scale and 63–69% of continuation tokens are already greedy-correct, which is what the decoder in §VII actually exploits.

a) *A second family confirms the tokenizer is the cause.*

To check that this is a property of the *tokenizer* and not of the languages, we repeat the measurement on XGLM-564M [22], a different family whose SentencePiece tokenizer was trained with heavy multilingual (including Indic) coverage. It fragments Indic words far less, and the first-token share rises in lockstep (Table V): a Tamil word is 2.5 tokens instead of 9.6, and 57% of its information is now in the first token instead of 8%. The phenomenon tracks fragmentation, not script: a model whose tokenizer spends vocabulary on Indic behaves, for Tamil and Hindi, much more like it does for English. The low first-token share under byte-level BPE is therefore a cost of the tokenizer, and the internal correlate of the sequence-length unfairness of Petrov et al. [3].

V. THE WORD IS READABLE BEFORE IT IS WRITTEN

If the word’s information is not in the first token, is it anywhere by the time the model is about to write the word? We train a linear probe on the pre-word hidden state—the state at the last context token, before any of the word is emitted—to predict the upcoming word among the 1,000 most

	Tokens / word		First-token share	
	Qwen3	XGLM	Qwen3	XGLM
Tamil	9.59	2.54	7.7%	56.5%
Hindi	4.82	1.46	24.9%	81.6%
English	1.14	1.21	94.7%	92.8%

TABLE V
CROSS-FAMILY CHECK (QWEN3-0.6B BYTE-LEVEL BPE VS XGLM-564M SENTENCEPIECE). THE LESS A TOKENIZER FRAGMENTS A WORD, THE MORE OF THE WORD’S INFORMATION SITS IN ITS FIRST TOKEN. THE EFFECT IS TOKENIZER-DRIVEN, NOT LANGUAGE-INTRINSIC.

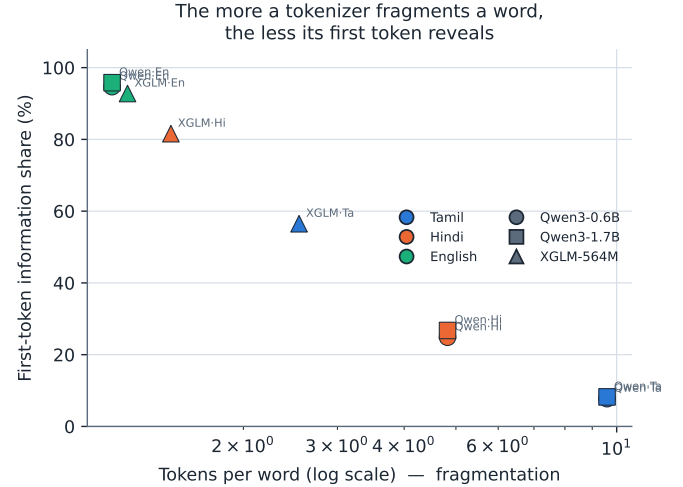


Fig. 5. Across all nine model×language points, the first-token share falls monotonically as the tokenizer fragments the word more. The phenomenon tracks fragmentation, not script: XGLM’s Indic-aware tokenizer (few tokens per word) keeps the information in the first token, like English.

frequent multi-token words. Because byte-level BPE gives almost all Tamil words the same first token, we cannot restrict candidates by first token as one would in English; we predict the whole word identity. We compare against two baselines: the most-frequent-word rate (dictionary prior) and a shuffled-label control task [5]. We also split test accuracy by whether the model itself goes on to spell the word correctly. Formally, with $h_{\text{pre}}^{(\ell)}$ the pre-word state and $\mathcal{C}$ the candidate words, the probe is the regularized multinomial logistic regression

$$\min_{W,b} \sum_{(h,y)} -\log \text{softmax}(Wh_{\text{pre}}^{(\ell)} + b)_y + \lambda \|W\|_2^2, \quad (4)$$

trained on the fit split and evaluated on held-out words; the shuffled-label control refits the same model on permuted labels.

Table VI shows the probe beats both baselines at every layer and for both models. Tamil reaches 14.5% (0.6B) and 16.6% (1.7B) against a 3.2% dictionary prior and a 1.5% control; Hindi reaches 39.8/43.5% against 24.4% and 15.3%. Readability rises through roughly the first quarter of the network and then plateaus. Crucially, on words the model later spells correctly the probe is far more accurate—34.6% vs. 12.0% on misspelled words for Tamil 0.6B, 69.2% vs. 17.8% for Hindi—so the decodable signal tracks the model’s own success, not just corpus frequency. Every probe–control gap is large relative to seed variation (3 seeds): Tamil +12.5 points

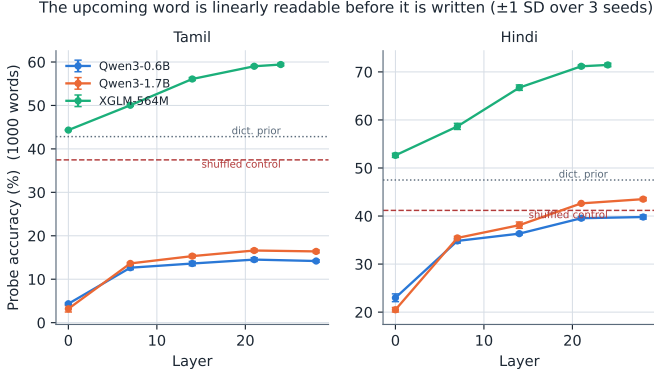


Fig. 6. Probe accuracy by layer with ± 1 SD over three seeds, against the dictionary prior and the shuffled-label control. Readability rises through the first quarter of the network and then plateaus, far above both baselines.

Layer	Probe	Most-freq.	Control	Chance
<i>Tamil</i> (1000 classes, 4,515 held-out)				
7	12.7 / 13.6	3.2	1.5	1.0
14	13.6 / 15.3	3.2	1.5	1.0
21	14.5 / 16.6	3.2	1.5	1.0
28	14.2 / 16.4	3.2	1.7	1.0
<i>Hindi</i> (1000 classes, 16,370 held-out)				
7	34.8 / 35.4	24.4	14.1	1.2
14	36.3 / 38.1	24.4	14.5	1.2
21	39.5 / 42.6	24.4	14.8	1.2
28	39.8 / 43.5	24.4	15.3	1.2

TABLE VI

PRE-WORD PLAN PROBE (% ACCURACY, 0.6B / 1.7B WHERE TWO NUMBERS). THE PROBE BEATS BOTH THE DICTIONARY PRIOR AND THE SHUFFLED-LABEL CONTROL AT EVERY LAYER, AND READABILITY SHARPENS WITH MODEL SIZE.

at 43 SD and Hindi +24.7 at 58 SD on 0.6B, rising to 46 and 83 SD on 1.7B.

a) *Does the probe exceed the model’s own predictions?:* The dictionary and control baselines are context-blind, so a natural worry is that the probe succeeds merely because the hidden state sees the context. We therefore add a *logit-lens* baseline that uses the model’s own computation: within each first-token group we rank the candidate words by the model’s teacher-forced likelihood of the whole word given the context (the strongest single readout of what the model itself predicts). On a matched subset (150 examples, groups capped at 20 candidates) this baseline reaches 16.7% (Tamil) and 46.7% (Hindi) top-1, against majority 2.0%/29.3%—i.e. *comparable to the pre-word probe* (14.5%/39.8% on the full eval), not below it. We read this honestly: the probe does not extract word identity *beyond* what the model itself predicts; rather, the pre-word state already contains, in linearly decodable form, essentially the same word information the model surfaces through its own distribution. This is consistent with—and strengthens—our causal finding below that the pre-word signal is re-derived predictive context (“breadcrumbs”), not a privileged stored plan the probe alone can see.

b) *As the word unfolds.:* We then probe the state available after $j = 1, \dots, 6$ tokens of the word have been written, against only those candidate words consistent with the j -token

Tokens written	State	Prefix-only	Gap
<i>Tamil</i> (1.7B)			
1	17.3	4.0	+13.2
2	32.0	14.7	+17.4
3	53.1	34.2	+18.9
4	63.0	48.5	+14.5
<i>Hindi</i> (1.7B)			
1	43.6	24.5	+19.1
2	67.9	41.2	+26.7
3	80.0	61.3	+18.6
4	85.5	75.1	+10.5

TABLE VII

WORD IDENTITY AS THE WORD UNFOLDS (% ACCURACY, QWEN3-1.7B). THE HIDDEN STATE IDENTIFIES THE WORD EARLIER THAN ITS WRITTEN PREFIX DOES.

prefix, and compare to a prefix-only dictionary that guesses the most frequent completion of that prefix. Table VII reports the gap. The model’s state stays well ahead of the written prefix throughout the word: for Hindi the advantage peaks at +23 points (0.6B) / +27 points (1.7B) after two tokens, and closes only as the prefix itself becomes unambiguous near the word’s end. The model “knows” more about the word than it has committed to paper.

VI. THE WORD IS NECESSARY BUT NOT LOCALIZED AT THE PRE-WORD POSITION

A probe shows a feature is *present* and linearly decodable, not that the model *uses* it or that it is *stored* where the probe reads. We run two causal tests, both on natural text, to decide this (Fig. 7).

a) *Activation patching.:* We mine pairs of real contexts (A, B) in which the model confidently and greedily writes two different multi-token words X and Y that share their first token. We then run context B while overwriting the residual stream at B ’s pre-word position, at one layer, with A ’s activation from the same position, and measure how far the next-token logit difference moves toward X (“effect”, normalized so 1.0 = fully A -like), and whether the model then spells X out in full (“transfer”). As a control we patch the first *in-word* position instead. Writing $m = \ell_{x_2} - \ell_{y_2}$ for the next-token logit difference between X ’s and Y ’s second tokens, with m_A, m_B its values in the two unpatched contexts and m_P its value after patching A ’s layer- ℓ state into B , the normalized effect is

$$\Delta_{A \rightarrow B}^{(\ell)} = \frac{m_P - m_B}{m_A - m_B}, \quad (5)$$

so $\Delta = 0$ leaves B unchanged and $\Delta = 1$ is fully A -like.

The pre-word patch is weak at every layer and every scale: the median effect peaks around 0.08 (Tamil) and 0.18 (Hindi) in the middle layers, and the donor word is spelled out in at most 4–6% of pairs. Patching the first *in-word* position instead is far stronger *at every matched layer*: at layer 18 of 28, for instance, its median effect is 0.53 (Tamil) and 0.57 (Hindi)—roughly nine and seven times the pre-word value at the same layer—and it transfers the full donor word in up to 44% (Tamil) / 38% (Hindi) of pairs (Figure 8). We deliberately

Figure B3. Triangulating the pre-word position: readable, necessary, but not localized

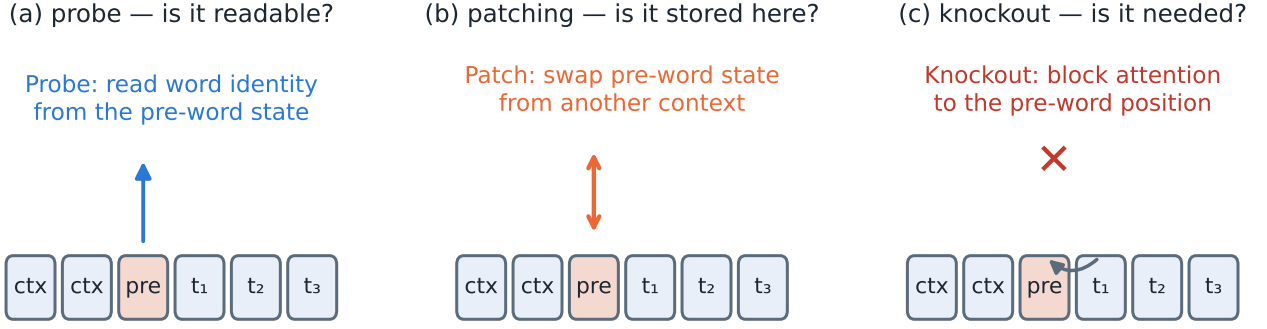
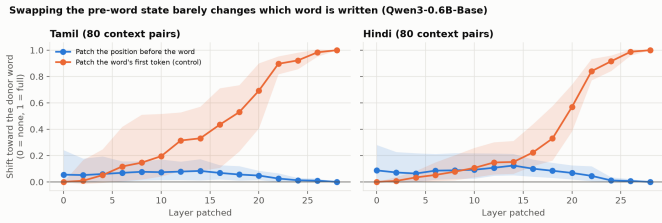
Fig. 7. Three interventions on the pre-word position. The probe tests whether the upcoming word is *readable* there; activation patching whether it is *stored* there (swap the state from another context); attention knockout whether it is *needed* there (block the word’s attention to it).

Fig. 8. Activation patching, Qwen3-0.6B. Swapping the pre-word state (blue) barely changes which word is written; swapping the first in-word position (orange control) dominates the decision across the mid-to-late layers (the exact 1.0 at the final layer is constructional; see text). The word is not localized before it is written.

read the contrast at the shared mid-to-late layers rather than at the endpoints: at the *final* layer the in-word control reaches an effect of exactly 1.00 and the pre-word effect falls to 0 *by construction*—a position’s last-layer residual feeds only its own logits and is never read by the later word position—so those endpoints carry no causal information and no claim rests on them. The mid-network comparison is what matters, and it is unambiguous: the choice of word is *not* sitting at the pre-word position waiting to be executed; it is assembled over the course of the word, and the position that accumulates the causal signal is the one *inside* the word, in the late-middle layers. This contradicts the “words before tokens” hypothesis in its strong form: across matched middle layers the in-word control transfers the donor word far more often than the pre-word patch, with no pair showing the reverse (McNemar $p < 10^{-4}$ for every model and language).

b) Attention knockout.: Patching asks whether the pre-word content *determines* the word; knockout asks whether it is *needed* at all. For words the model spells correctly, we block attention from the word’s own positions to one earlier position—at every layer, via an explicit additive mask—and measure the change in the continuation’s log-probability.

Lang.	Blocked position	Δ bits	Spelled
Tamil	pre-word	−2.5	25%
Tamil	token before that	−0.25	59%
Tamil	random earlier	−0.02	61%
Hindi	pre-word	−1.9	54%
Hindi	token before that	−0.03	82%
Hindi	random earlier	−0.01	81%

TABLE VIII
ATTENTION KNOCKOUT, QWEN3-1.7B (300 CORRECTLY-SPELLED WORDS EACH). THE PRE-WORD POSITION IS A NECESSARY, LOCAL RELAY; ITS NEIGHBOURS ARE NOT.

Table VIII shows a sharp, local effect: blocking the pre-word position costs 2.3–2.5 bits on Tamil and 1.8–1.9 on Hindi, and drops correct spelling to 25–28% (Tamil) and 52–54% (Hindi). Blocking the token just before it, or a random earlier context token, costs essentially nothing (≤ 0.25 bits) and leaves spelling almost intact. The pre-word cost dwarfs the random-position control on a paired test (Wilcoxon $p < 10^{-30}$ for both languages and models); the bootstrap 95% CI on the mean cost is, e.g., $[-2.82, -2.23]$ bits for Tamil on 1.7B versus $[-0.05, +0.01]$ for a random position. The result is stable across three random resamplings of the test words: the pre-word cost is 2.3 ± 0.1 bits (Tamil) and 1.9 ± 0.1 (Hindi) on 0.6B, and 2.4 ± 0.3 and 1.9 ± 0.1 on 1.7B, while the random-position cost stays within noise of zero in every seed.

c) Reading.: The two results are complementary, not contradictory. The pre-word position is a *necessary relay*—the word’s own tokens attend back to it and lose about two bits of certainty when that link is cut—yet its content does not *carry* the identity of the word, since replacing it with the content from a different word’s context does not redirect the output. The most consistent interpretation is that the pre-word position passes forward local, boundary-level information (where the word starts, immediate context) that the spelling needs, while

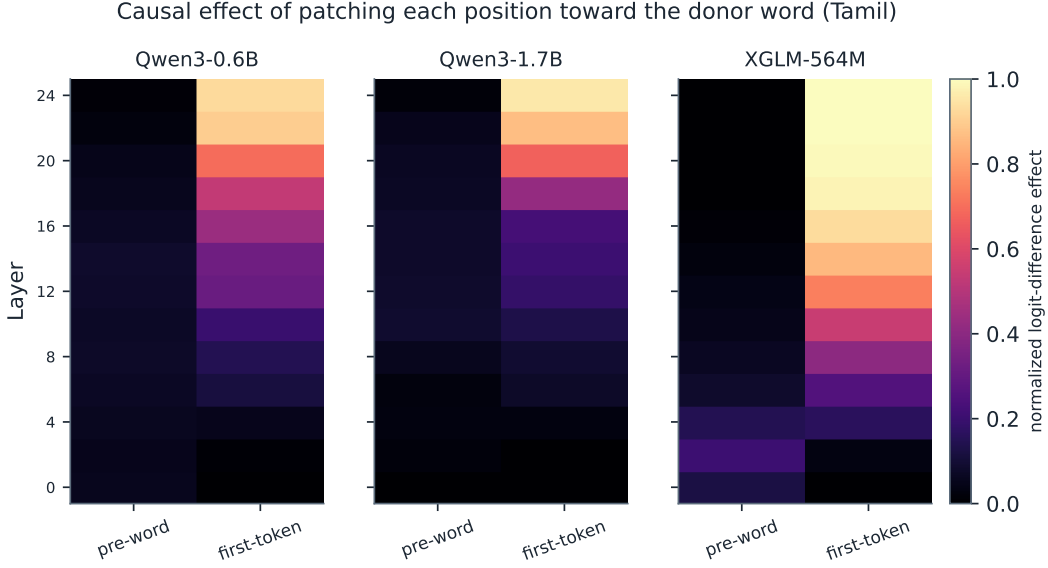


Fig. 9. Per-layer causal effect of patching each position toward the donor word (Tamil), for all three models. The pre-word column stays dark (no effect); the first-token column lights up in the late layers, where the word is actually assembled. The pattern is identical across families.

the word’s identity is recomputed from the whole context at each interior step. This is why a probe can read the word there (the information is linearly present in the shared context representation) even though patching cannot move it (it is not a localized variable). It also predicts the ceiling we hit next: a drafter that reads one early state cannot be certain of a word that the model itself has not yet fixed.

d) The mechanism replicates on a second family.: Run on XGLM-564M, the same picture holds and is if anything sharper: patching the pre-word position transfers the donor word in 0% of pairs for both languages while the first-token control reaches 74% (Tamil) and 89% (Hindi), and knockout of the pre-word position costs 1.4 bits of continuation certainty against ≤ 0.06 for a random position. The probe likewise beats its baselines (Tamil 59% vs. a 38% control and 43% dictionary prior). “Readable, necessary, but not localized” is therefore not an artifact of one tokenizer or one model family. Figure 11 summarizes the effect sizes: across all six model×language settings the first-token—pre-word transfer gap and the pre-word knockout cost both have 95% bootstrap confidence intervals that exclude zero by a wide margin.

VII. A LOSSLESS WORD-LEVEL SPECULATIVE DECODER

The mechanism says a word is partly-but-not-fully determined in advance. We now ask how much latency that buys back, under the hard constraint that the output must not change at all.

a) Decoder.: At each step we take the target model’s own greedy token t . If t begins a new word, we ask a drafter for the remaining tokens of that word given the current hidden state; we then run a single forward pass on $[t]$ + draft and accept drafted tokens for as long as they equal the model’s own greedy argmax, cropping the KV cache on the first rejection. Because acceptance is defined by the target model’s argmax, the output is token-for-token identical to plain greedy decoding

Algorithm 1 Lossless word-level speculative decoding

Require: target model M , drafter D , prompt x

```

1: while not done do
2:    $t \leftarrow \arg \max M(x)$  ▷ target’s own greedy token
3:   if  $t$  starts a new word then
4:      $d \leftarrow D(h_{\text{cur}})$  ▷ propose  $(d_1, \dots, d_k)$ 
5:     run one pass of  $M$  on  $[t, d_1, \dots, d_k]$ 
6:      $j \leftarrow \text{largest prefix with } d_i = \arg \max M(x, t, d_{\leq i}) \forall i \leq j$ 
7:     append  $t, d_1, \dots, d_j$ ; crop KV cache to the accepted length
8:   else
9:     append  $t$ 
10:  end if
11: end while
12: guarantee: output  $\equiv$  plain greedy decoding

```

by construction; we also verify this empirically, with the only exceptions being floating-point ties (discussed under “On ‘lossless’” below). Algorithm 1 states the loop, and Fig. 12 shows it schematically.

b) Drafters.: All drafters read frozen features and are fit by convex optimization on the fit split, in under two minutes each on the laptop. A *word head* is a multinomial logistic regression over a lexicon (plus an OTHER class) from a standardized hidden state; we train one on the pre-word state and one on the current in-word state (§VI motivates reading the freshest state). A *dictionary* drafter is the context-blind baseline: the most frequent lexicon word consistent with the written prefix. *Token-ahead heads* are K linear heads à la Medusa/multi-token prediction [8, 9] that predict the next K tokens from the current state and token embedding over a small output vocabulary; they need no lexicon, so they are not capped by lexicon coverage (only 39% of held-out Tamil word

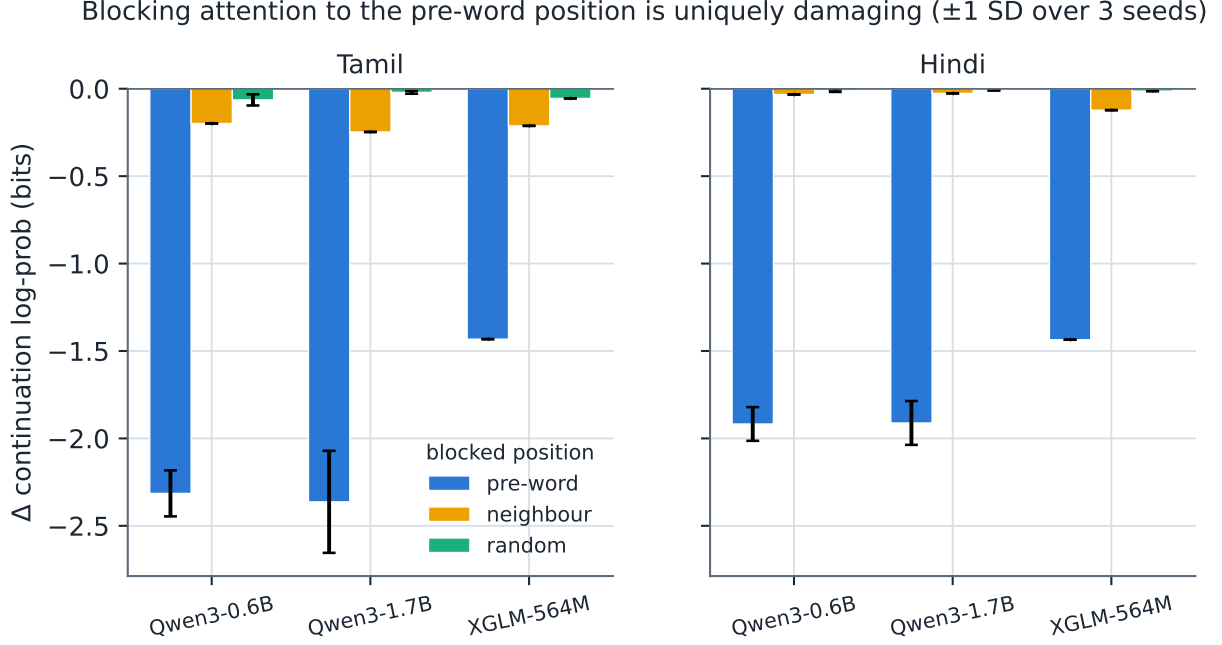


Fig. 10. Change in the word’s continuation log-probability when attention to one earlier position is blocked (± 1 SD over three seeds). Only the pre-word position matters, across all models and both languages.

Drafter	Tamil		Hindi	
	tok/fwd	wall	tok/fwd	wall
plain greedy	1.00	$\times 1.00$	1.00	$\times 1.00$
dictionary	1.50	$\times 1.43$	1.47	$\times 1.74$
pre-word head	1.64	$\times 1.49$	1.70	$\times 1.92$
current head	1.71	$\times 1.56$	1.70	$\times 1.86$
token-ahead heads	1.85	$\times 1.73$	1.60	$\times 1.75$
word + ahead	2.00	$\times 1.67$	1.75	$\times 1.83$

TABLE IX

REAL LOSSLESS GREEDY DECODING ON QWEN3-1.7B (12 HELD-OUT PROMPTS, 64 NEW TOKENS EACH). OUTPUT MATCHED PLAIN GREEDY ON EVERY PROMPT UP TO FLOATING-POINT TIES (SEE TEXT). “TOK/FWD” IS ACCEPTED TOKENS PER FORWARD PASS; “WALL” IS WALL-CLOCK SPEEDUP ON THE M1 GPU.

occurrences on the head lexicon). We also chain them: use the word head where it has a candidate, fall back to token-ahead heads otherwise. Figure 13 shows both head architectures.

c) Results.: Table IX reports real decoding on the 1.7B model; output matched plain greedy up to floating-point ties on all prompts, and the same held for 0.6B. The best drafter reaches 2.0 accepted tokens per forward pass on Tamil and gives a wall-clock speedup of $\times 1.67$ – $\times 1.73$ on Tamil and $\times 1.83$ – $\times 1.92$ on Hindi, even though every drafter runs in pure Python on top of the model. Reading the model’s *current* state beats the context-blind dictionary consistently, and the gain over plain decoding is larger on 1.7B than on 0.6B (e.g. Tamil word+ahead, $\times 1.46$ to $\times 1.67$ wall-clock), because the fixed cost of the linear heads is amortized against a more expensive base forward pass. The full trend across all three sizes, including where it plateaus at 4B, is reported in Table X. Bootstrap 95% CIs over prompts put the best drafter’s tokens per forward pass clearly above one on every model

and language (e.g. Tamil [1.88, 2.20] on 1.7B); the wall-clock speedup is noisier on throttled MPS but positive on 9–11 of 12 prompts. The emulated upper bound (a drafter that is always right) is $\times 2.5$ fewer passes on Tamil, so a meaningful gap remains; two things limit us, both predicted by the mechanism: lexicon coverage is low under heavy inflection, and the word is not fixed early (§VI), so an early draft is often wrong and only partially accepted. Token-ahead heads, which re-read the state every step and need no lexicon, are the single best stand-alone drafter on Tamil for exactly this reason.

d) The gain is proportional to fragmentation.: Running the same lossless decoder on XGLM-564M, whose tokenizer barely fragments Indic, the best drafter reaches only 1.1 tokens per forward pass and no wall-clock win (Tamil $\times 1.20$, Hindi $\times 0.97$)—there is almost no within-word structure left to draft. This is the point: the method recovers latency exactly where the tokenizer destroys it (Qwen3 Tamil, $\times 1.67$) and does nothing where tokenization is already efficient. The decoder is a remedy for fragmentation, and its benefit scales with how badly a model’s tokenizer fragments the script (Fig. 15).

A. TokenLift: a learned word-scoped draft head

The convex drafters above are deliberately simple, and two facts cap them: a lexicon head cannot draft a word it has never seen (held-out Tamil lexicon coverage is only 39%), and a token-ahead head predicts each position from the *same* pre-word state, so its drafts drift as the word gets longer. Both limits are symptoms of drafting without a model of the word’s own unfolding. We therefore add a learned, autoregressive head in the style of EAGLE [10], scoped to a single word, and release it as a small tool, TOKENLIFT.

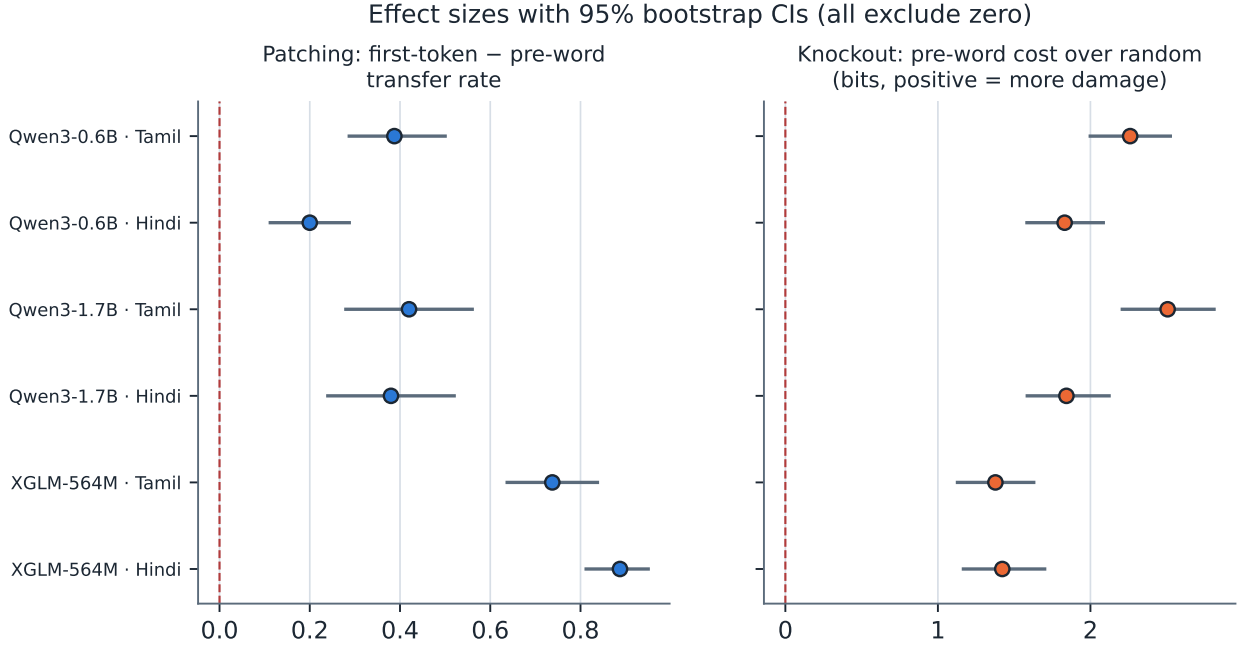


Fig. 11. Effect sizes with 95% bootstrap confidence intervals. Left: patching transfer gap (first-token minus pre-word). Right: knockout cost of the pre-word position over a random position (bits). Every interval excludes zero.

Figure B4. Draft-verify loop; acceptance is defined by the target's own arg max, so output is provably unchanged

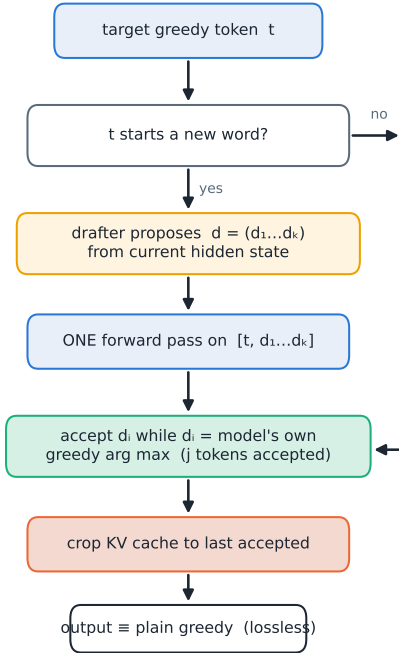


Fig. 12. The draft-verify loop. Acceptance is defined by the target's own arg max, so the output distribution is provably unchanged.

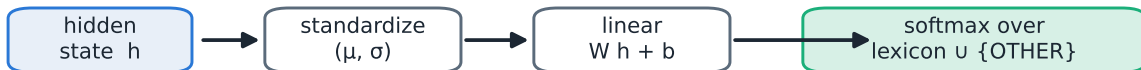
a) Head. The head is a two-layer residual MLP that reads a standardized hidden state and the current token embedding and predicts (i) the *next hidden state* and (ii) a distribution over the next token (a K -word lexicon plus OTHER and STOP classes). At draft time it is run autoregressively on its own

predicted feature: draft a token, feed the predicted feature and that token's embedding back in, and continue until it emits STOP or the word ends. Verification is unchanged— $[t] + \text{draft}$ is checked in one target pass—so the output is still identical to plain greedy.

b) Training. Teacher forcing a feature-autoregressive head is a known trap: at test time the head consumes its own slightly-wrong features (exposure bias), and a head trained only on gold features drifts. We train with *scheduled-sampling rollouts* [11]: with a probability annealed from 0 to 0.5 over training, each step consumes the head's own predicted (detached) feature instead of the gold one. This is the same remedy, specialized to a word-scoped head, that HASS and EAGLE-3 apply to full-sequence draft heads under the names harmonized training and the “training-time test” [? ?]; our contribution is the word scoping and the Indic setting, not the self-feeding objective itself. With it, the head learns to recover from its own errors. The loss is a cross-entropy on the next token plus a feature-regression term; the whole head fits on frozen features in a few minutes on the laptop. The ablation is clearest on the quality signal, tokens per forward pass: the teacher-forced and rollout heads sit at essentially the same wall-clock on Tamil ($\times 1.40$ vs $\times 1.41$, within MPS timing noise), but the rollout head accepts markedly more tokens per forward (below)—the wall-clock parity reflects the head's tiny fixed cost, not equal drafting quality.

c) Comparison to standard self-speculation. The obvious same-tokenizer baseline is ordinary speculative decoding with a smaller model of the same family as the drafter. We run it on the 4B target with Qwen3-0.6B-Base as the draft model ($K=5$, greedy, same prompts; textbook accept/reject, [12, 13]), and it is *stronger* than our word-scoped head on

Figure B5. Both drafters are linear, fit by convex optimization on frozen features

Word head**Token-ahead heads (Medusa-style)**Fig. 13. The two drafters. The word head is a standardized linear softmax over a lexicon; the token-ahead heads are K parallel linear maps predicting the next K tokens. Both are convex to fit on frozen features.

Model	Decoder	tok/fwd	accept	wall
0.6B, Ta	convex (word+ahead)	1.89	18.7%	$\times 1.12$
	TokenLift head	2.15	21.2%	$\times 1.41$
0.6B, Hi	convex (word+ahead)	1.68	24.9%	$\times 1.15$
	TokenLift head	1.82	30.2%	$\times 1.40$
1.7B, Ta	convex (word+ahead)	2.00	21.3%	$\times 1.40$
	TokenLift head	2.26	23.6%	$\times 1.84$
1.7B, Hi	convex (word+ahead)	1.79	28.7%	$\times 1.73$
	TokenLift head	1.86	30.0%	$\times 2.10$
4.0B, Ta	convex (word+ahead)	2.03	22.0%	$\times 1.66$
	TokenLift head	2.23	24.5%	$\times 1.73$
4.0B, Hi	convex (word+ahead)	1.82	30.9%	$\times 1.97$
	TokenLift head	1.98	38.0%	$\times 2.09$

TABLE X

THE LEARNED WORD-SCOPED HEAD VS. THE BEST CONVEX DRAFTER, REAL LOSSLESS GREEDY DECODING (24 HELD-OUT PROMPTS, 64 NEW TOKENS EACH), ACROSS THREE MODEL SIZES. THE HEAD WINS ON EVERY MODEL AND LANGUAGE IN ACCEPTED TOKENS PER FORWARD PASS. THE WALL-CLOCK PAYOFF IS MODEST AT 0.6B ($\times 1.1$ – $\times 1.4$) AND LARGE AT 1.7B–4B ($\times 1.7$ – $\times 2.1$): THE FIXED COST OF THE HEAD IS AMORTIZED AGAINST A MORE EXPENSIVE BASE FORWARD PASS, SO THE METHOD IS MOST USEFUL EXACTLY AT DEPLOYABLE MODEL SIZES. WALL-CLOCK IS MEASURED ON A THROTTLED M1 GPU AND IS NOISIER THAN TOKENS-PER-FORWARD (E.G. TAMIL 1.7B VS. 4B), WHICH IS THE HARDWARE-INDEPENDENT QUALITY SIGNAL AND STAYS ABOVE THE CONVEX BASELINE THROUGHOUT.

accepted tokens per forward: 3.40 vs. 2.23 on Tamil and 3.23 vs. 1.98 on Hindi (Table XI; both exact against plain 4B greedy on the verified prompts). This is expected, and we state it plainly: a full draft model is unrestricted—it drafts across word boundaries and uses far more capacity than a two-layer head—so it accepts more per pass. Word scoping is a *restriction*, and it costs acceptance.

What the word-scoped head buys instead is that it needs *no separate draft model*. The trained head is 10.7M parameters (21 MB in float16, just 0.27% of the 4B target), read off

Decoder (4B target)	Ta tok/fwd	Hi tok/fwd	extra mem (fp16)
standard self-spec (0.6B draft)	3.40	3.23	1.19 GB
TokenLift head (no draft model)	2.23	1.98	21 MB

TABLE XI

STANDARD SELF-SPECULATIVE DECODING (A FULL 0.6B DRAFT MODEL) VS. THE DRAFT-MODEL-FREE WORD-SCOPED HEAD, ON THE 4B TARGET, 12 HELD-OUT PROMPTS, 64 NEW TOKENS, $K=5$, FLOAT16. BOTH ARE LOSSLESS (GREEDY OUTPUT MATCHES PLAIN 4B GREEDY; VERIFIED EXACTLY ON HELD-OUT PROMPTS). THE FULL DRAFT MODEL ACCEPTS MORE PER PASS, BUT AT $56\times$ THE AUXILIARY MEMORY (10.7M-PARAMETER HEAD VS. A 596M DRAFT MODEL) AND ONLY WHEN A SAME-TOKENIZER DRAFT MODEL EXISTS; THE HEAD NEEDS NO SECOND MODEL.

the target’s own features and trained in minutes on a laptop. Standard self-speculation must instead keep a full 0.6B model resident (596M parameters, 1.19 GB in float16— $56\times$ the head, about a 15% memory surcharge on the 4B target) and run it K times per step, and it presupposes that a suitable smaller model with the *same* tokenizer exists—often it does not, especially for the smallest deployable Indic models. So the trade is explicit: self-speculation accepts more per pass (3.3 vs. 2.2) at $56\times$ the auxiliary memory and a draft-model dependency; TokenLift recovers a useful part of the headroom for a 21 MB head and no second model. TokenLift should therefore be read as a lightweight, single-model accelerator for when a draft model is unavailable or memory is tight, not as a method that beats draft-model speculative decoding where that is an option.

d) Lossless speculative sampling.: Greedy verification only preserves the arg-max path. For temperature $\tau > 0$ we keep losslessness in the distributional sense with the standard speculative-sampling rule [12, 13]: accept a drafted token with probability $\min(1, p_i(d_i)/q_i(d_i))$ and otherwise resample from the normalized residual $(p_i - q_i)_+$, which provably leaves the target’s sampling distribution unchanged. The draft distribution

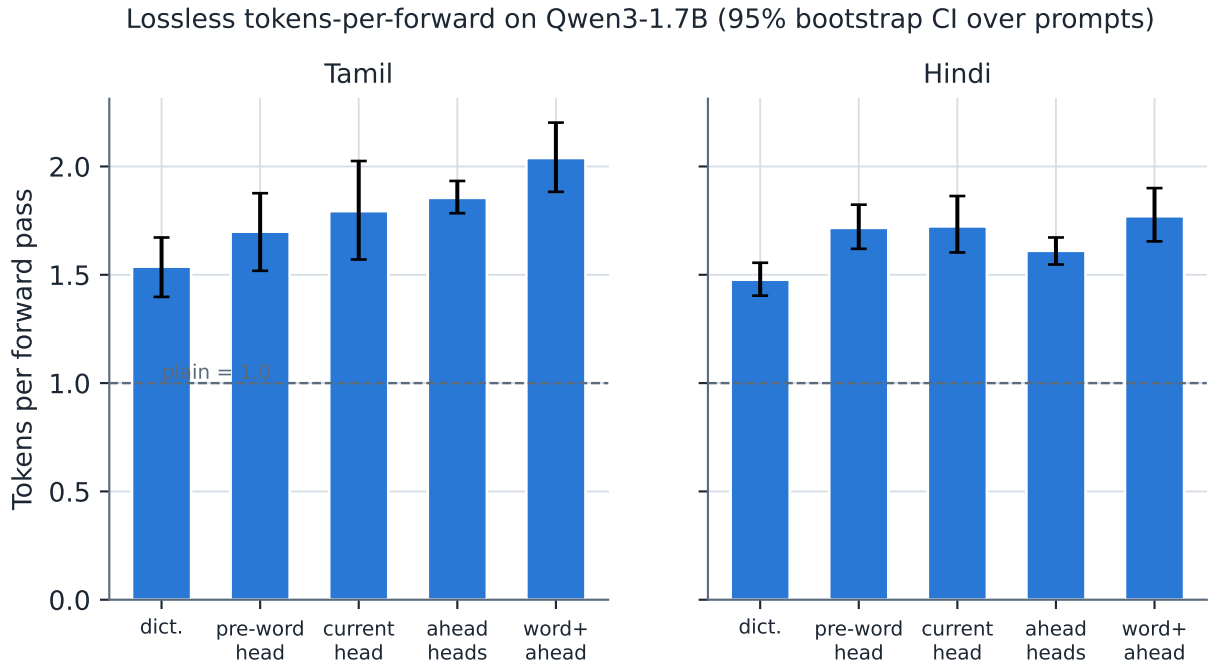


Fig. 14. Accepted tokens per forward pass by drafter on Qwen3-1.7B, with 95% bootstrap confidence intervals over prompts. Every drafter is strictly above the plain-decoding baseline of one.

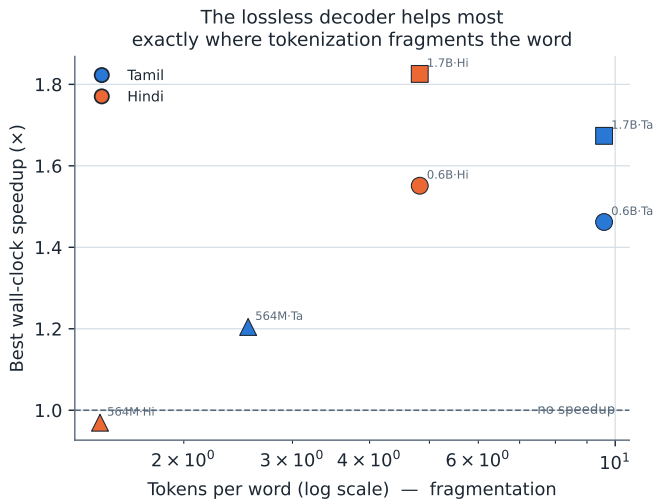


Fig. 15. Best wall-clock speedup against fragmentation, across models and languages. The lossless decoder helps most exactly where the tokenizer fragments the word; where tokenization is already efficient (XGLM) it is roughly neutral.

q_i must be full-vocabulary, so we send the head’s predicted feature through the *target model’s own* final norm and LM head (EAGLE-style) rather than the head’s small classifier. The acceleration survives, and at moderate temperature improves: on Tamil the head reaches 2.32 tokens per forward at $\tau=0.7$ and 2.09 at $\tau=1.0$; on Hindi 1.85 and 1.74 (Table XII). Higher-temperature acceptance is limited only by genuine target entropy, not by the method.

	Tamil		Hindi	
Temperature	tok/fwd	accept	tok/fwd	accept
$\tau = 0.7$	2.32	28.2%	1.85	28.4%
$\tau = 1.0$	2.09	24.7%	1.74	27.9%

TABLE XII
LOSSLESS SPECULATIVE *sampling* WITH THE TOKENLIFT HEAD (QWEN3-0.6B-BASE, 12 HELD-OUT PROMPTS, 64 NEW TOKENS). THE ACCEPT/REJECT RULE PRESERVES THE TARGET’S τ -SAMPLED DISTRIBUTION BY CONSTRUCTION; THE NUMBERS ARE SPEED, NOT QUALITY.

e) On “lossless”.: Greedy acceleration is identical to plain greedy *by construction*, and is bit-identical on the large majority of prompts. (The decoder runs reported here use float16 on the laptop GPU, where such ties are more frequent than in float32; the released tool defaults to float32 for strict bit-identity.) The rare divergences we see are not approximation error: they occur only at floating-point ties, where two tokens share the top logit to within $\sim 10^{-2}$ and the arg-max order flips with batch shape—so plain greedy decoding is itself order-dependent at exactly those positions. The method changes the batch shape of the forward pass (one pass over $[t] + \text{draft}$ rather than several single-token passes), which is why ties surface; it never changes the model’s distribution. Comprehension is untouched throughout: the model only ever reads its own original sub-word tokens.

f) A released tool.: We package the head, both decoders, and a headroom diagnostic as TOKENLIFT (`pip install; MIT`). The diagnostic estimates achievable speed-up for any model and script from two measured quantities—tokens per word and the fraction of continuation tokens the model pre-

dicts greedily—so a practitioner can decide whether to train a head without running one. We validate it against the measured oracle upper bound across three models and two languages (six configurations, including the low-fragmentation XGLM cases it should rate poorly): the predicted tokens-per-forward correlates with the measured oracle at Pearson $r = 0.98$ with a mean absolute error of 0.10 tokens per forward, and it correctly separates the high-headroom Qwen3 Indic cases (predicted 2.1–2.6) from the low-headroom XGLM cases (predicted 1.2–1.4). The diagnostic is thus a reliable, training-free screen for where the method will pay off. TOKENLIFT is, to our knowledge, the first speculative decoder scoped to the *word* as the unit that tokenization fragments, and the first to target Indic generation specifically.

VIII. DISCUSSION AND LIMITATIONS

a) What the picture is.: Byte-level tokenization does not just make Indic words longer; it moves their information out of the first token and spreads it across encoding-internal continuation tokens, most of which still carry a residual decision. The upcoming word is linearly readable from the model’s state before and during writing, and that readability tracks the model’s own accuracy, but the word is not a localized variable that is set once and executed—it is re-derived from context at each step, with the pre-word position acting as a necessary relay rather than a store. A lossless decoder built on this reads the freshest state, drafts a whole word, and recovers a good part of the latency lost to fragmentation.

b) Limitations.: (i) The mechanistic analysis covers three models—two Qwen3 sizes ($\leq 1.7\text{B}$) and a second family, XGLM-564M—and the decoder is evaluated on Qwen3-0.6B, 1.7B and 4B for Tamil and Hindi. Two tokenizer families and $\leq 4\text{B}$ parameters is a narrow sweep: 7–8B models do not fit this single-laptop setup, and other families (Gemma, Llama) are future work. (ii) Greedy decoding is exact. Speculative *sampling* preserves the target distribution by the standard accept/reject rule, but we benchmark it only on Qwen3-0.6B (Table XII). (iii) Wall-clock numbers are batch-size-1 on Metal/MPS and include Python-side drafter overhead; on CPU the extra draft tokens are not free and the same decoder is slower than plain decoding. (iv) Patching uses 50–80 context pairs and knockout 300 words; we report bootstrap 95% CIs and paired tests, and seed-robustness over the random selection, but the knockout is a whole-layer attention block, not a per-head path patch, so “necessary relay” is a coarse claim pending finer analysis. (v) Wikipedia text only, and the 20231101 dump may overlap the models’ training data. We note the scope of that threat precisely: it can inflate predictability-dependent quantities (the greedy-continuation rate, and hence drafter *acceptance* and speed-up), but it cannot affect correctness—every decoder here is lossless by construction, so contamination changes how fast we go, never the output—nor the tokenizer-intrinsic descriptive findings (tokens-per-word and the first-token information share are deterministic functions of the tokenizer and UTF-8, independent of any training overlap). As partial evidence that the effects are not a single model’s memorization artifact,

every finding replicates across a held-out-by-sequence split and across two model families with different training data (Qwen3 and XGLM). A dedicated post-cutoff, non-Wikipedia Indic holdout remains the proper test and is future work. (vi) The causal reading in §VI is an interpretation consistent with three experiments, not a validated circuit.

c) Threats to validity.: *Construct:* the “word” is defined by space/word-break segmentation over decoded token bytes, and “information” by teacher-forced surprisal; both are standard but imperfect proxies. *Internal:* the mechanism conclusion rests on converging evidence from a probe, patching, and knockout rather than a single test, and we report bootstrap CIs and paired nonparametric tests (§VI, Fig. 11) with seed-robustness for the knockout; the whole-layer knockout is still coarser than per-head path patching. *External:* mechanistic analysis on three models up to 1.7B parameters and decoder results up to 4B, two tokenizer families, Wikipedia text, and speculative sampling benchmarked on one model; larger models, more domains, and more languages remain to be tested. *Reproducibility:* every number is produced from released JSON by a report script, and the full pipeline runs on a single consumer laptop.

IX. CONCLUSION

We asked where a word’s information lives when a tokenizer has shattered it into many pieces. In byte-level Indic-script models the answer is: almost nowhere in the first token, linearly readable but not localized before the word is written, and re-derived from context as the word unfolds. The practical consequence is a lossless word-level speculative decoder that fits on a laptop and returns up to twice the tokens per forward pass. We release the code, data pipeline, and the raw results behind every figure and table so that both the measurements and the decoder can be reproduced and extended to other scripts and model families.

REPRODUCIBILITY

All code (segmentation, corpus pass, probes, patching, knockout, heads, and the lossless decoder), the data-building scripts, unit tests, and the raw JSON results that generate every number and figure in this paper are available at <https://github.com/rochit72/research-paper>. Every table is produced from `results/` by a report script; no number here is typed by hand. The full pipeline runs on a single MacBook M1 Pro.

ACKNOWLEDGMENT

The authors thank the open-source communities behind PyTorch, Hugging Face Transformers, and the Qwen and XGLM model releases, and Wikimedia for the Wikipedia corpora used in this study.

REFERENCES

- [1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI technical report*, 2019.

- [2] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.
- [3] A. Petrov, E. La Malfa, P. H. S. Torr, and A. Bibi, “Language model tokenizers introduce unfairness between languages,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [4] O. Ahia, S. Kumar, H. Gonen, J. Kasai, D. R. Mortensen, N. A. Smith, and Y. Tsvetkov, “Do all languages cost the same? tokenization in the era of commercial language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [5] J. Hewitt and P. Liang, “Designing and interpreting probes with control tasks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [6] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, “Locating and editing factual associations in gpt,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [7] K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt, “Interpretability in the wild: a circuit for indirect object identification in gpt-2 small,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [8] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple llm inference acceleration framework with multiple decoding heads,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [9] F. Gloeckle, B. Youbi Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve, “Better & faster large language models via multi-token prediction,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [10] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle: Speculative sampling requires rethinking feature uncertainty,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [11] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [12] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *International Conference on Machine Learning (ICML)*, 2023.
- [13] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating large language model decoding with speculative sampling,” *arXiv preprint arXiv:2302.01318*, 2023.
- [14] M. Stern, N. Shazeer, and J. Uszkoreit, “Blockwise parallel decoding for deep autoregressive models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [15] E. Yi, T. Kim, H. Jeung, D.-S. Chang, and S.-Y. Yun, “Towards fast multilingual llm inference: Speculative decoding and specialized drafters,” *arXiv preprint arXiv:2406.16758*, 2024.
- [16] G. Alain and Y. Bengio, “Understanding intermediate layers using linear classifier probes,” in *International Conference on Learning Representations (ICLR) Workshop*, 2017.
- [17] N. Belrose, Z. Furman, L. Smith, D. Halawi, I. Ostrovsky, L. McKinney, S. Biderman, and J. Steinhardt, “Eliciting latent predictions from transformers with the tuned lens,” *arXiv preprint arXiv:2303.08112*, 2023.
- [18] N. Elhage, N. Nanda, C. Olsson *et al.*, “A mathematical framework for transformer circuits,” *Transformer Circuits Thread*, 2021.
- [19] N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora, “Localizing model behavior with path patching,” *arXiv preprint arXiv:2304.05969*, 2023.
- [20] K. Pal, J. Sun, A. Yuan, B. C. Wallace, and D. Bau, “Future lens: Anticipating subsequent tokens from a single hidden state,” in *Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL)*, 2023.
- [21] W. Wu, J. X. Morris, and L. Levine, “Do language models plan ahead for future tokens?” in *Conference on Language Modeling (COLM)*, 2024.
- [22] X. V. Lin, T. Mihaylov, M. Artetxe *et al.*, “Few-shot learning with multilingual generative language models,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.